Healthcare Orchestration & Integration Platform (Laravel)

Single Source of Truth (SSoT) — Build Spec + Standards + Runbooks + API Catalog

Project codename: MedFlow

Last updated: 2026-02-28 (Asia/Tashkent)

---

0. Purpose

Build an enterprise-grade, multi-tenant, event-driven **Healthcare Workflow & Integration Platform**. - Manage patients, providers, clinics, appointments, treatment plans, lab orders, prescriptions, billing, insurance claims. - Orchestrate workflows **state machines**. - Integrate with Uzbekistan services: **Payme, Click, Uzum**, SMS providers (**Eskiz, Play Mobile, Textup**), **Telegram Bot**, **Email**, **Google Auth**, plus optional **MyID** and **E-IMZO**. - Provide **tons of endpoints**, complete **API documentation**, and complete **developer documentation**. - Built **AI-agent friendly** (Codex-ready): small files, strict patterns, explicit rules, checklists, and a single reference.

---

1. Technology Stack (Pinned)

> Versions pinned at time of writing; patch upgrades allowed only through approved dependency update process.

Core

- PHP 8.5.3
- Laravel 12.53.0
- Composer 2.x (lockfile enforced)

Database + Cache

- PostgreSQL 18.3
- Redis 8.6.0 (GA) **with TLS enabled**

Messaging / Streaming

- Apache Kafka 4.2.0

API Documentation

- OpenAPI 3.1.1
- Swagger UI 5.3 1.2

Testing & Quality

- Pest 4.2.0
- PHPStan (max level; pinned in composer)
- Psalm (strict; pinned in composer)
- Laravel Pint (pinned; project config)

Observability

- Sentry PHP SDK 4.2 1.0
- OpenTelemetry Collector 0.1 4 6.1
- Prometheus 3.1 0.0
- Grafana 1 2.3.4
- Elastic Stack (Elasticsearch/Kibana) 9.3.1

Runtime / Deployment

- Docker + Docker Compose
- Nginx (or Caddy) reverse proxy
- Horizon/Queue workers (if using Laravel queues for internal async) + Kafka consumers

---

2. Non-Negotiable Engineering Rules (AI-Agent Friendly)

These rules are enforced by CI and must be followed by humans and AI agents.

2.1 Small Files & Single Responsibility

- **Max file length:** 250 lines (soft), 400 lines (hard). Exceeding hard limit fails CI.
- **Max method length:** 40 lines.
- **Max class public methods:** 12.
- **Cyclomatic complexity:** keep $\leq$ 10 per method.

2.2 Layering & Dependency Rules (Clean Architecture)

Allowed dependencies: `Presentation` → may depend `Application` interfaces/DTOs. - `Application` → may depe `Domain` (entities, value objects, domain services, `Domain`). → depends on nothing else (no Laravel, no Eloque `Infrastructure` - → implements interfaces defined in `Application` / `Domain`.

Forbidden: - Domain importing Laravel classes. - Controllers using Eloquent models directly for bu logic. - Controllers making external HTTP calls directly. - Cross-module imports without public contracts.

2.3 Dependency Injection Everywhere

- No `new` in business code except factories.
- All external services accessed through interfaces and injected.
- All integrations behind adapters.

2.4 Patterns Required

This project intentionally uses many patterns; use them consistently: - Repository, Unit of Work (t boundary), Factory, Builder - Strategy, Adapter, Facade (internal), Decorator - Observer (events), Med (event bus), Command - Chain of Responsibility (pipelines), Specification - State pattern (state mac CQRS (Commands/Queries) at application layer - Outbox pattern for Kafka publishing

2.5 "No Magic" Rule

- No hidden global helpers for business logic.
- Each use-case must be implemented as an **Application Command/Query handler**.

2.6 Security-by-Default

- **Database ports MUST NOT be exposed** in docker-compose.
- All secrets via env / secret manager, never committed.
- TLS for Redis and external integrations.
- OAuth tokens encrypted at rest.

2.7 Documentation Required for Every Change

Every PR must include: - Updated OpenAPI (or explicitly "no API change") - Updated ADR if decision - Updated tests - Updated module docs if behavior changed

---

3. Product Requirements

3.1 Tenancy

- Multi-tenant by `tenant_id` (clinic network / enterprise group).
- Tenant isolation at DB row-level via:
- mandatory `tenant_id` column,
- global query scope in infrastructure only,
- request context sets tenant.

3.2 Roles & Permissions

Roles (baseline): - SuperAdmin, TenantAdmin, ClinicAdmin, Doctor, Nurse, Receptionist, LabTech, BillingAgent, Patient.

Permissions: - Fine-grained action permissions, policy-driven. - Permission caching per user with ev based invalidation.

3.3 Audit

- Full audit trail for regulated objects:
- who, what, when, before/after, request-id/correlation-id.
- Write-once storage semantics for audit events.

---

4. Architecture

4.1 High-Level

- HTTP API (Laravel) is the edge.
- Domain is pure.
- Application orchestrates use-cases.
- Infrastructure provides adapters:
- DB repositories
- Kafka producer/consumer
- Redis caches
- External integrations

4.2 Module Boundaries

Each module has its own domain, application handlers, contracts, and infrastructure. Modules: Identity & Access (IAM) 2. Tenant & Clinic Management 3. Patient 4. Provider (Doctor/Nurse) Scheduling (Appointment) 6. Treatment Plans 7. Lab Orders & Results 8 Prescriptions 9 Billing & Payments 10. Insurance Claims 11. Notifications 12. Integrations Hub 13. Audit & Compliance 14. Observability

4.3 Directory Structure

```
app/
  Modules/
    Scheduling/
      Domain/
      Application/
      Infrastructure/
      Presentation/
    Billing/
    ...
  Shared/
    Domain/
    Application/
    Infrastructure/
```

```
bootstrap/
config/
database/
docs/
  api/
  dev/
  adr/
```

---

5. Domain Model (Core Aggregates)

5.1 Aggregates

- Tenant
- Clinic
- User
- Patient
- Provider
- Appointment
- TreatmentPlan
- LabOrder
- Prescription
- Invoice
- Payment
- InsuranceClaim
- Notification

5.2 Value Objects (examples)

- Money (amount, currency)
- PhoneNumber (E.164, country)
- EmailAddress
- NationalId (optional)
- AppointmentSlot
- Address
- ExternalReference (provider + id)

5.3 Domain Events (examples)

- AppointmentScheduled
- AppointmentConfirmed
- AppointmentCanceled
- TreatmentPlanApproved
- LabOrderCreated
- LabResultReceived
- InvoiceIssued

- PaymentCaptured
- ClaimSubmitted
- ClaimApproved

---

6 . State Machines

6 . 1 Appointment State Machine

States: `draft` - `scheduled` - `confirmed` - `checked_in` - `in_progress` - `completed` - side branches: `canceled`, `no_show`, `rescheduled`

Guards: - cannot confirm past appointment - cannot check-in without confirmation (unless admin ov cannot complete unless in_progress

Transitions produce events for Kafka + audit.

6 . 2 Insurance Claim State Machine

- `draft` → `submitted` → `under_review` → `approved|denied` → `paid` (if approved)

6 . 3 Payment State Machine

- `initiated` → `pending` → `captured|failed|canceled` → `refunded` (optional)

6 . 4 Implementation Contract

- All state transitions go through **Application handlers**.
- State machine logic lives in Domain (pure), invoked via Application.

---

7 . Messaging (Kafka)

7 . 1 Topics

- `medflow.appointments.v1`
- `medflow.treatments.v1`
- `medflow.billing.v1`
- `medflow.claims.v1`
- `medflow.notifications.v1`
- `medflow.audit.v1`
- `medflow.integrations.v1`

7.2 Event Envelope (Standard)

Fields: - `event_id` (UUID) `event_type` `occurred_at` `tenant_id` `correlation_id` - `causation_id` - `actor` (user_id/service) - `payload` (schema per event) - `version`

7.3 Delivery Guarantees

- At-least-once processing.
- Idempotency required for consumers.

7.4 Outbox Pattern (Required)

- Business transaction writes domain changes + outbox record in same DB transaction.
- Outbox relay publishes to Kafka.
- Relay retries with backoff.

---

8. Caching (Redis)

8.1 What to Cache

- permission sets
- provider availability
- clinic settings
- integration token metadata
- rate limit buckets
- frequently queried reference data

8.2 Patterns

- Cache-aside with explicit invalidation on domain events.
- Tagged caching by tenant.

8.3 Security

- Redis TLS + auth
- keys include tenant prefix

---

9. Database (PostgreSQL)

9.1 Principles

- DB schema is authoritative and versioned by migrations.
- Use UUIDs for public identifiers.
- Separate internal numeric ids allowed but not exposed.

9.2 Constraints & Indexing

- Foreign keys enforced.
- Partial indexes for common filters (tenant_id + status + dates).
- Full-text search for patient/provider directories.

---

10. Integrations Hub (Uzbekistan + General)

10.1 Integration Framework

All integrations follow the same patt`Contract` (interface) in Applicatio`Adapter` in Infrastructure - `Client` wraps HTTP `Authenticator` handles auth `RetryPolicy` + `CircuitBreaker` `WebhookVerifier` for inboun`Mapper` converts external payloads → internal DTOs

10.2 Payment Integrations

- Payme
- Click
- Uzum

Common features: - create payment - query status - handle callbacks/webhooks - reconcile - re supported)

10.3 Identity

- Google OAuth (login)
- Optional: MyID (KYC / identity verification)

10.4 SMS Providers

- Eskiz
- Play Mobile
- TextUp

Strategy pattern:`SmsProvider` interface - dynamic routing rules: - failover provider order - per-ten provider - per-message type (OTP vs marketing)

10.5 Telegram Bot

- patient reminders
- clinic broadcast
- support channel

10.6 Email

- transactional email service (SMTP or provider)
- templates versioned

10.7 Additional Uzbekistan Integrations (Optional)

These are designed as plug-ins; implement when you have credentials: - E-IMZO e-signature work Global ID Gate / other local identity aggregators - Tax/receipt systems (if applicable) - Local map providers (if needed)

---

11. API Design Standards

11.1 REST + Action Endpoints

- Resource-first routes.
- Action routes for transitions (confirm, cancel, approve).

11.2 Versioning

- URL: `/api/v1/...`
- Event versioning separate from API version.

11.3 Error Model

Standard error: - `code`, `message`, `details`, `trace_id`, `correlation_id`

11.4 Pagination

- cursor pagination preferred.

11.5 Idempotency

- Idempotency keys for:
- payment creation
- appointment scheduling
- webhook processing

---

12. Endpoint Catalog (High-Level)

This is the initial catalog. Each endpoint is documented in OpenAPI with full schemas an examples.

12.1 Auth & Identity

- POST `/api/v1/auth/login`
- POST `/api/v1/auth/logout`
- POST `/api/v1/auth/refresh`
- GET `/api/v1/auth/me`
- GET `/api/v1/auth/google/redirect`
- GET `/api/v1/auth/google/callback`

12.2 Tenants & Clinics

- CRUD tenants
- CRUD clinics
- clinic settings

12.3 Users & RBAC

- CRUD users
- roles
- permissions

12.4 Patients

- CRUD patients
- search
- attach documents

12.5 Providers

- CRUD providers
- availability
- calendars

12.6 Scheduling (Appointments)

- CRUD appointments
- actions: schedule, confirm, check-in, start, complete, cancel, no-show, reschedule

12.7 Treatments

- CRUD treatment plans
- actions: approve, pause, resume, finish

12.8 Labs

- CRUD lab orders
- receive results (webhook)

12.9 Prescriptions

- CRUD prescriptions
- actions: issue, cancel

12.10 Billing & Payments

- invoices
- payments: initiate, status, refund
- reconcile

12.11 Insurance

- claims
- actions: submit, review, approve/deny, mark-paid

12.12 Notifications

- email/sms/telegram templates
- send test

12.13 Integrations

- manage credentials per tenant
- webhook endpoints
- health checks

12.14 Audit

- query audit events

12.15 Observability

- health, readiness, liveness
- metrics endpoint

Note: The full endpoint matrix (200+ routes) is `/docs/api/endpoint-matrix.md` section below with route naming conventions, request/response schemas, and state transitions.

---

13. Testing Strategy (100% Coverage Target)

13.1 Test Types

- Unit tests (Domain + pure Application logic)
- Feature tests (HTTP API)
- Integration tests (DB, Redis, Kafka test containers)
- Contract tests (OpenAPI schema validation)

- Consumer-driven contract tests for integrations (where possible)

13.2 Coverage Policy

- Coverage measured for:
- Domain
- Application
- Infrastructure critical paths
- 100% for Domain + Application required.

13.3 Test Data

- Builders + Mother objects.
- Deterministic time.

---

14. Observability & Logging (Best Practices)

14.1 Logging

- Structured JSON logs.
- Correlation IDs:
- `X-Request-Id`
- `X-Correlation-Id`
- Log levels standardized.

14.2 Tracing

- OpenTelemetry instrumentation for:
- inbound HTTP
- outbound HTTP integrations
- DB queries (sampled)
- Kafka produce/consume

14.3 Errors

- Sentry for exception aggregation.

14.4 Metrics

- Prometheus metrics for:
- request latency
- queue lag
- kafka consumer lag
- integration error rates
- cache hit ratio

14.5 Central Logs

- Elastic for log indexing and dashboards.

---

15. Security

- OAuth 2 / JWT
- Rate limiting per tenant and per IP
- Webhook signature verification for all inbound callbacks
- Encrypt integration secrets at rest
- PII handling: field-level encryption for sensitive fields
- Audit logging mandatory

---

16. CI/CD

16.1 Pipeline Gates

- lint (Pint)
- static analysis (PHPStan + Psalm)
- unit tests
- feature tests
- integration tests (optional on main)
- OpenAPI validation
- coverage thresholds
- architecture rules (layering, file size)

16.2 Releases

- semantic versioning
- changelog generation

---

17. Docker & Local Dev

17.1 Compose Rules

- DB ports **NOT exposed**.
- services on a private network.
- only API gateway ports exposed.

17.2 Services

- app

- nginx
- postgres
- redis
- kafka
- otel-collector
- prometheus
- grafana
- elasticsearch
- kibana

---

18. AI Agent / Codex Playbook (How to Work in This Repo)

18.1 Golden Path: Adding a New Feature

1. Choose module.
2. Add domain types (entity/value objects/events).
3. Add application command/query + handler.
4. Add interface contracts.
5. Implement infrastructure adapters.
6. Add controller + request DTO.
7. Add OpenAPI definitions + examples.
8. Add tests (unit + feature + integration if needed).
9. Update docs (endpoint matrix + module docs).

18.2 PR Checklist

- [ ] All CI gates pass
- [ ] No file > 400 lines
- [ ] No domain imports Laravel
- [ ] OpenAPI updated
- [ ] Tests added
- [ ] Observability: logs/metrics/traces updated if needed

---

19. Documents Included (Virtual File List)

Everything below is included in this single SSoT document; in the repo these are separat
markdown files under `/docs`.

`/docs/dev/`

- `architecture.md` (this doc sections 4–5)
- `coding-standards.md` (section 2)
- `testing.md` (section 13)

- `observability.md` (section 14)
- `security.md` (section 15)
- `ci-cd.md` (section 16)
- `local-dev.md` (section 17)
- `ai-agent-playbook.md` (section 18)

`/docs/api/`

- `openapi-guidelines.md`
- `endpoint-matrix.md` (expanded route inventory)
- `error-catalog.md`
- `webhooks.md`

`/docs/adr/`

- `000-template.md`
- `001-clean-architecture.md`
- `002-kafka-outbox.md`
- `003-redis-caching.md`
- `004-integrations-framework.md`
- `005-observability-stack.md`

---

20. Next Steps (Implementation Roadmap)

1. Bootstrap Laravel 12 + module structure + shared kernel.
2. Implement IAM + tenancy context + audit.
3. Implement Kafka outbox + consumer framework.
4. Implement integration framework + sandbox stubs.
5. Implement scheduling + appointment state machine.
6. Add billing + Payme/Click/Uzum.
7. Add notifications with SMS failover + Telegram.
8. Add full observability stack.
9. Expand endpoint matrix and finalize OpenAPI.

Appendix A — Endpoint Matrix (Full Inventory v1)

Target inventory: 280 **endpoints** (within requested 200–400). This matrix is the authoritative route catalog. Each endpoint maps to exactly one Application handler (Command/Query). Each route must be documented in OpenAPI with schemas and examples.

A.0 Conventions

- Base prefix: `/api/v1`

- Route format: `METHOD /path` → `UseCase` → `Module`
- `:action` routes are explicit workflow/state transitions.
- Bulk actions use `/bulk`.
- Exports use `/export`.
- Search endpoints use `/search` or `?q=`.
- Webhooks live under `/webhooks/{provider}`.
- Admin-only endpoints live under `/admin/*`.

A.1 Auth & Identity (16)

- POST `/auth/login` → `LoginCommand` → IAM
- POST `/auth/logout` → `LogoutCommand` → IAM
- POST `/auth/refresh` → `RefreshTokenCommand` → IAM
- GET `/auth/me` → `GetMeQuery` → IAM
- POST `/auth/password/forgot` → `RequestPasswordResetCommand` → IAM
- POST `/auth/password/reset` → `ResetPasswordCommand` → IAM
- POST `/auth/mfa/setup` → `SetupMfaCommand` → IAM
- POST `/auth/mfa/verify` → `VerifyMfaCommand` → IAM
- POST `/auth/mfa/disable` → `DisableMfaCommand` → IAM
- GET `/auth/google/redirect` → `GoogleRedirectQuery` → IAM
- GET `/auth/google/callback` → `GoogleCallbackCommand` → IAM
- POST `/auth/sessions` → `ListSessionsQuery` → IAM
- DELETE `/auth/sessions/{sessionId}` → `RevokeSessionCommand` → IAM
- POST `/auth/api-keys` → `CreateApiKeyCommand` → IAM
- GET `/auth/api-keys` → `ListApiKeysQuery` → IAM
- DELETE `/auth/api-keys/{keyId}` → `RevokeApiKeyCommand` → IAM

A.2 Tenants (12)

- GET `/tenants` → `ListTenantsQuery` → Tenancy
- POST `/tenants` → `CreateTenantCommand` → Tenancy
- GET `/tenants/{tenantId}` → `GetTenantQuery` → Tenancy
- PATCH `/tenants/{tenantId}` → `UpdateTenantCommand` → Tenancy
- DELETE `/tenants/{tenantId}` → `DeleteTenantCommand` → Tenancy
- POST `/tenants/{tenantId}:activate` → `ActivateTenantCommand` → Tenancy
- POST `/tenants/{tenantId}:suspend` → `SuspendTenantCommand` → Tenancy
- GET `/tenants/{tenantId}/usage` → `GetTenantUsageQuery` → Tenancy
- GET `/tenants/{tenantId}/limits` → `GetTenantLimitsQuery` → Tenancy
- PUT `/tenants/{tenantId}/limits` → `UpdateTenantLimitsCommand` → Tenancy
- GET `/tenants/{tenantId}/settings` → `GetTenantSettingsQuery` → Tenancy
- PUT `/tenants/{tenantId}/settings` → `UpdateTenantSettingsCommand` → Tenancy

A.3 Clinics & Locations (26)

- GET `/clinics` → `ListClinicsQuery` → Clinics
- POST `/clinics` → `CreateClinicCommand` → Clinics
- GET `/clinics/{clinicId}` → `GetClinicQuery` → Clinics
- PATCH `/clinics/{clinicId}` → `UpdateClinicCommand` → Clinics
- DELETE `/clinics/{clinicId}` → `DeleteClinicCommand` → Clinics
- POST `/clinics/{clinicId}:activate` → `ActivateClinicCommand` → Clinics
- POST `/clinics/{clinicId}:deactivate` → `DeactivateClinicCommand` → Clinics
- GET `/clinics/{clinicId}/settings` → `GetClinicSettingsQuery` → Clinics
- PUT `/clinics/{clinicId}/settings` → `UpdateClinicSettingsCommand` → Clinics
- GET `/clinics/{clinicId}/departments` → `ListDepartmentsQuery` → Clinics
- POST `/clinics/{clinicId}/departments` → `CreateDepartmentCommand` → Clinics
- GET `/clinics/{clinicId}/departments/{deptId}` → `GetDepartmentQuery` → Clinics
- PATCH `/clinics/{clinicId}/departments/{deptId}` → `UpdateDepartmentCommand` → Clinics
- DELETE `/clinics/{clinicId}/departments/{deptId}` → `DeleteDepartmentCommand` → Clinics
- GET `/clinics/{clinicId}/rooms` → `ListRoomsQuery` → Clinics
- POST `/clinics/{clinicId}/rooms` → `CreateRoomCommand` → Clinics
- PATCH `/clinics/{clinicId}/rooms/{roomId}` → `UpdateRoomCommand` → Clinics
- DELETE `/clinics/{clinicId}/rooms/{roomId}` → `DeleteRoomCommand` → Clinics
- GET `/clinics/{clinicId}/work-hours` → `GetClinicWorkHoursQuery` → Clinics
- PUT `/clinics/{clinicId}/work-hours` → `UpdateClinicWorkHoursCommand` → Clinics
- GET `/clinics/{clinicId}/holidays` → `ListClinicHolidaysQuery` → Clinics
- POST `/clinics/{clinicId}/holidays` → `CreateClinicHolidayCommand` → Clinics
- DELETE `/clinics/{clinicId}/holidays/{holidayId}` → `DeleteClinicHolidayCommand` → Clinics
- GET `/locations/cities` → `ListCitiesQuery` → Clinics
- GET `/locations/districts` → `ListDistrictsQuery` → Clinics
- GET `/locations/search` → `SearchLocationsQuery` → Clinics

A.4 Users, Roles, Permissions (38)

- GET `/users` → `ListUsersQuery` → IAM
- POST `/users` → `CreateUserCommand` → IAM
- GET `/users/{userId}` → `GetUserQuery` → IAM
- PATCH `/users/{userId}` → `UpdateUserCommand` → IAM
- DELETE `/users/{userId}` → `DeleteUserCommand` → IAM
- POST `/users/{userId}:activate` → `ActivateUserCommand` → IAM
- POST `/users/{userId}:deactivate` → `DeactivateUserCommand` → IAM
- POST `/users/{userId}:lock` → `LockUserCommand` → IAM
- POST `/users/{userId}:unlock` → `UnlockUserCommand` → IAM

- POST `/users/{userId}:reset-password` → `AdminResetPasswordCommand` → IAM
- GET `/users/{userId}/roles` → `ListUserRolesQuery` → IAM
- PUT `/users/{userId}/roles` → `SetUserRolesCommand` → IAM
- GET `/users/{userId}/permissions` → `GetUserPermissionsQuery` → IAM
- POST `/users:bulk-import` → `BulkImportUsersCommand` → IAM
- POST `/users/bulk` → `BulkUpdateUsersCommand` → IAM
- GET `/roles` → `ListRolesQuery` → IAM
- POST `/roles` → `CreateRoleCommand` → IAM
- GET `/roles/{roleId}` → `GetRoleQuery` → IAM
- PATCH `/roles/{roleId}` → `UpdateRoleCommand` → IAM
- DELETE `/roles/{roleId}` → `DeleteRoleCommand` → IAM
- GET `/roles/{roleId}/permissions` → `ListRolePermissionsQuery` → IAM
- PUT `/roles/{roleId}/permissions` → `SetRolePermissionsCommand` → IAM
- GET `/permissions` → `ListPermissionsQuery` → IAM
- GET `/permissions/groups` → `ListPermissionGroupsQuery` → IAM
- GET `/rbac/audit` → `GetRbacAuditQuery` → IAM
- GET `/profiles/me` → `GetMyProfileQuery` → IAM
- PATCH `/profiles/me` → `UpdateMyProfileCommand` → IAM
- POST `/profiles/me/avatar` → `UploadMyAvatarCommand` → IAM
- GET `/profiles/{userId}` → `GetProfileQuery` → IAM
- PATCH `/profiles/{userId}` → `UpdateProfileCommand` → IAM
- GET `/devices` → `ListDevicesQuery` → IAM
- POST `/devices` → `RegisterDeviceCommand` → IAM
- DELETE `/devices/{deviceId}` → `DeregisterDeviceCommand` → IAM
- GET `/security/events` → `ListSecurityEventsQuery` → IAM
- GET `/security/events/{eventId}` → `GetSecurityEventQuery` → IAM
- POST `/security/ip-allowlist` → `UpdateIpAllowlistCommand` → IAM
- GET `/security/ip-allowlist` → `GetIpAllowlistQuery` → IAM
- POST `/security/sessions:revoke-all` → `RevokeAllSessionsCommand` → IAM

A.5 Patients (30)

- GET `/patients` → `ListPatientsQuery` → Patient
- POST `/patients` → `CreatePatientCommand` → Patient
- GET `/patients/{patientId}` → `GetPatientQuery` → Patient
- PATCH `/patients/{patientId}` → `UpdatePatientCommand` → Patient
- DELETE `/patients/{patientId}` → `DeletePatientCommand` → Patient
- GET `/patients/search` → `SearchPatientsQuery` → Patient
- GET `/patients/{patientId}/summary` → `GetPatientSummaryQuery` → Patient
- GET `/patients/{patientId}/timeline` → `GetPatientTimelineQuery` → Patient
- GET `/patients/{patientId}/contacts` → `ListPatientContactsQuery` → Patient
- POST `/patients/{patientId}/contacts` → `CreatePatientContactCommand` → Patient

- PATCH `/patients/{patientId}/contacts/{contactId}` → `UpdatePatientContactCommand` → Patient
- DELETE `/patients/{patientId}/contacts/{contactId}` → `DeletePatientContactCommand` → Patient
- GET `/patients/{patientId}/insurance` → `ListPatientInsuranceQuery` → Insurance
- POST `/patients/{patientId}/insurance` → `AttachPatientInsuranceCommand` → Insurance
- DELETE `/patients/{patientId}/insurance/{policyId}` → `DetachPatientInsuranceCommand` → Insurance
- GET `/patients/{patientId}/documents` → `ListPatientDocumentsQuery` → Patient
- POST `/patients/{patientId}/documents` → `UploadPatientDocumentCommand` → Patient
- GET `/patients/{patientId}/documents/{docId}` → `GetPatientDocumentQuery` → Patient
- DELETE `/patients/{patientId}/documents/{docId}` → `DeletePatientDocumentCommand` → Patient
- GET `/patients/{patientId}/consents` → `ListPatientConsentsQuery` → Patient
- POST `/patients/{patientId}/consents` → `CreatePatientConsentCommand` → Patient
- POST `/patients/{patientId}/consents/{consentId}:revoke` → `RevokePatientConsentCommand` → Patient
- GET `/patients/{patientId}/tags` → `ListPatientTagsQuery` → Patient
- PUT `/patients/{patientId}/tags` → `SetPatientTagsCommand` → Patient
- POST `/patients:bulk-import` → `BulkImportPatientsCommand` → Patient
- POST `/patients/bulk` → `BulkUpdatePatientsCommand` → Patient
- GET `/patients/export` → `ExportPatientsQuery` → Patient
- GET `/patients/{patientId}/external-refs` → `ListPatientExternalRefsQuery` → Integrations
- POST `/patients/{patientId}/external-refs` → `AttachPatientExternalRefCommand` → Integrations
- DELETE `/patients/{patientId}/external-refs/{refId}` → `DetachPatientExternalRefCommand` → Integrations

A. 6 Providers (Doctors/Nurses) & Availability (3 4)

- GET `/providers` → `ListProvidersQuery` → Provider
- POST `/providers` → `CreateProviderCommand` → Provider
- GET `/providers/{providerId}` → `GetProviderQuery` → Provider
- PATCH `/providers/{providerId}` → `UpdateProviderCommand` → Provider
- DELETE `/providers/{providerId}` → `DeleteProviderCommand` → Provider
- GET `/providers/search` → `SearchProvidersQuery` → Provider
- GET `/providers/{providerId}/profile` → `GetProviderProfileQuery` → Provider
- PATCH `/providers/{providerId}/profile` → `UpdateProviderProfileCommand` → Provider
- GET `/providers/{providerId}/specialties` → `ListProviderSpecialtiesQuery` → Provider

- PUT `/providers/{providerId}/specialties` → `SetProviderSpecialtiesCommand` → Provider
- GET `/providers/{providerId}/licenses` → `ListProviderLicensesQuery` → Provider
- POST `/providers/{providerId}/licenses` → `AddProviderLicenseCommand` → Provider
- DELETE `/providers/{providerId}/licenses/{licenseId}` → `RemoveProviderLicenseCommand` → Provider
- GET `/providers/{providerId}/availability/rules` → `ListAvailabilityRulesQuery` → Scheduling
- POST `/providers/{providerId}/availability/rules` → `CreateAvailabilityRuleCommand` → Scheduling
- PATCH `/providers/{providerId}/availability/rules/{ruleId}` → `UpdateAvailabilityRuleCommand` → Scheduling
- DELETE `/providers/{providerId}/availability/rules/{ruleId}` → `DeleteAvailabilityRuleCommand` → Scheduling
- GET `/providers/{providerId}/availability/slots` → `GetAvailabilitySlotsQuery` → Scheduling
- POST `/providers/{providerId}/availability:rebuild-cache` → `RebuildAvailabilityCacheCommand` → Scheduling
- GET `/providers/{providerId}/calendar` → `GetProviderCalendarQuery` → Scheduling
- GET `/providers/{providerId}/calendar/export` → `ExportProviderCalendarQuery` → Scheduling
- GET `/providers/{providerId}/work-hours` → `GetProviderWorkHoursQuery` → Provider
- PUT `/providers/{providerId}/work-hours` → `UpdateProviderWorkHoursCommand` → Provider
- GET `/providers/{providerId}/time-off` → `ListTimeOffQuery` → Provider
- POST `/providers/{providerId}/time-off` → `CreateTimeOffCommand` → Provider
- PATCH `/providers/{providerId}/time-off/{timeOffId}` → `UpdateTimeOffCommand` → Provider
- DELETE `/providers/{providerId}/time-off/{timeOffId}` → `DeleteTimeOffCommand` → Provider
- GET `/specialties` → `ListSpecialtiesQuery` → Provider
- POST `/specialties` → `CreateSpecialtyCommand` → Provider
- PATCH `/specialties/{specialtyId}` → `UpdateSpecialtyCommand` → Provider
- DELETE `/specialties/{specialtyId}` → `DeleteSpecialtyCommand` → Provider
- GET `/provider-groups` → `ListProviderGroupsQuery` → Provider
- POST `/provider-groups` → `CreateProviderGroupCommand` → Provider
- PUT `/provider-groups/{groupId}/members` → `SetProviderGroupMembersCommand` → Provider

A. 7 Scheduling: Appointments (State Machine) (4 4)

Appointment resources

- GET `/appointments` → `ListAppointmentsQuery` → Scheduling

- POST `/appointments` → `CreateAppointmentCommand` → Scheduling
- GET `/appointments/{appointmentId}` → `GetAppointmentQuery` → Scheduling
- PATCH `/appointments/{appointmentId}` → `UpdateAppointmentCommand` → Scheduling
- DELETE `/appointments/{appointmentId}` → `DeleteAppointmentCommand` → Scheduling
- GET `/appointments/search` → `SearchAppointmentsQuery` → Scheduling
- GET `/appointments/export` → `ExportAppointmentsQuery` → Scheduling
- GET `/appointments/{appointmentId}/audit` → `GetAppointmentAuditQuery` → Audit

State transitions (actions)

- POST `/appointments/{appointmentId}:schedule` → `ScheduleAppointmentCommand` → Scheduling
- POST `/appointments/{appointmentId}:confirm` → `ConfirmAppointmentCommand` → Scheduling
- POST `/appointments/{appointmentId}:check-in` → `CheckInAppointmentCommand` → Scheduling
- POST `/appointments/{appointmentId}:start` → `StartAppointmentCommand` → Scheduling
- POST `/appointments/{appointmentId}:complete` → `CompleteAppointmentCommand` → Scheduling
- POST `/appointments/{appointmentId}:cancel` → `CancelAppointmentCommand` → Scheduling
- POST `/appointments/{appointmentId}:no-show` → `MarkNoShowCommand` → Scheduling
- POST `/appointments/{appointmentId}:reschedule` → `RescheduleAppointmentCommand` → Scheduling
- POST `/appointments/{appointmentId}:restore` → `RestoreAppointmentCommand` → Scheduling

Participants & notes

- GET `/appointments/{appointmentId}/participants` → `ListAppointmentParticipantsQuery` → Scheduling
- POST `/appointments/{appointmentId}/participants` → `AddAppointmentParticipantCommand` → Scheduling
- DELETE `/appointments/{appointmentId}/participants/{participantId}` → `RemoveAppointmentParticipantCommand` → Scheduling
- GET `/appointments/{appointmentId}/notes` → `ListAppointmentNotesQuery` → Scheduling
- POST `/appointments/{appointmentId}/notes` → `AddAppointmentNoteCommand` → Scheduling
- PATCH `/appointments/{appointmentId}/notes/{noteId}` → `UpdateAppointmentNoteCommand` → Scheduling
- DELETE `/appointments/{appointmentId}/notes/{noteId}` → `DeleteAppointmentNoteCommand` → Scheduling

Reminders & communications

- POST `/appointments/{appointmentId}:send-reminder` → `SendAppointmentReminderCommand` → Notifications
- POST `/appointments/{appointmentId}:send-confirmation` → `SendAppointmentConfirmationCommand` → Notifications

Recurrence (optional)

- POST `/appointments/{appointmentId}:make-recurring` → `MakeAppointmentRecurringCommand` → Scheduling
- POST `/appointments/recurrences/{recurrenceId}:cancel` → `CancelRecurrenceCommand` → Scheduling

Waitlist

- GET `/waitlist` → `ListWaitlistQuery` → Scheduling
- POST `/waitlist` → `AddToWaitlistCommand` → Scheduling
- DELETE `/waitlist/{entryId}` → `RemoveFromWaitlistCommand` → Scheduling
- POST `/waitlist/{entryId}:offer-slot` → `OfferWaitlistSlotCommand` → Scheduling

Bulk & ops

- POST `/appointments/bulk` → `BulkUpdateAppointmentsCommand` → Scheduling
- POST `/appointments:bulk-cancel` → `BulkCancelAppointmentsCommand` → Scheduling
- POST `/appointments:bulk-reschedule` → `BulkRescheduleAppointmentsCommand` → Scheduling
- POST `/appointments:rebuild-cache` → `RebuildSchedulingCacheCommand` → Scheduling

A.8 Treatment Plans & Encounters (32)

- GET `/treatment-plans` → `ListTreatmentPlansQuery` → Treatment
- POST `/treatment-plans` → `CreateTreatmentPlanCommand` → Treatment
- GET `/treatment-plans/{planId}` → `GetTreatmentPlanQuery` → Treatment
- PATCH `/treatment-plans/{planId}` → `UpdateTreatmentPlanCommand` → Treatment
- DELETE `/treatment-plans/{planId}` → `DeleteTreatmentPlanCommand` → Treatment
- GET `/treatment-plans/search` → `SearchTreatmentPlansQuery` → Treatment
- GET `/treatment-plans/{planId}/items` → `ListTreatmentItemsQuery` → Treatment
- POST `/treatment-plans/{planId}/items` → `AddTreatmentItemCommand` → Treatment
- PATCH `/treatment-plans/{planId}/items/{itemId}` → `UpdateTreatmentItemCommand` → Treatment
- DELETE `/treatment-plans/{planId}/items/{itemId}` → `RemoveTreatmentItemCommand` → Treatment

Treatment state actions

- POST `/treatment-plans/{planId}:approve` → `ApproveTreatmentPlanCommand` → Treatment
- POST `/treatment-plans/{planId}:start` → `StartTreatmentPlanCommand` → Treatment
- POST `/treatment-plans/{planId}:pause` → `PauseTreatmentPlanCommand` → Treatment
- POST `/treatment-plans/{planId}:resume` → `ResumeTreatmentPlanCommand` → Treatment
- POST `/treatment-plans/{planId}:finish` → `FinishTreatmentPlanCommand` → Treatment
- POST `/treatment-plans/{planId}:reject` → `RejectTreatmentPlanCommand` → Treatment

Encounters / visits

- GET `/encounters` → `ListEncountersQuery` → Treatment
- POST `/encounters` → `CreateEncounterCommand` → Treatment
- GET `/encounters/{encounterId}` → `GetEncounterQuery` → Treatment
- PATCH `/encounters/{encounterId}` → `UpdateEncounterCommand` → Treatment
- DELETE `/encounters/{encounterId}` → `DeleteEncounterCommand` → Treatment
- GET `/encounters/{encounterId}/diagnoses` → `ListDiagnosesQuery` → Treatment
- POST `/encounters/{encounterId}/diagnoses` → `AddDiagnosisCommand` → Treatment
- DELETE `/encounters/{encounterId}/diagnoses/{dxId}` → `RemoveDiagnosisCommand` → Treatment
- GET `/encounters/{encounterId}/procedures` → `ListProceduresQuery` → Treatment
- POST `/encounters/{encounterId}/procedures` → `AddProcedureCommand` → Treatment
- DELETE `/encounters/{encounterId}/procedures/{procId}` → `RemoveProcedureCommand` → Treatment
- GET `/encounters/export` → `ExportEncountersQuery` → Treatment
- POST `/encounters/bulk` → `BulkUpdateEncountersCommand` → Treatment

A.9 Labs: Orders & Results (28)

- GET `/lab-orders` → `ListLabOrdersQuery` → Lab
- POST `/lab-orders` → `CreateLabOrderCommand` → Lab
- GET `/lab-orders/{orderId}` → `GetLabOrderQuery` → Lab
- PATCH `/lab-orders/{orderId}` → `UpdateLabOrderCommand` → Lab
- DELETE `/lab-orders/{orderId}` → `DeleteLabOrderCommand` → Lab
- GET `/lab-orders/search` → `SearchLabOrdersQuery` → Lab
- POST `/lab-orders/{orderId}:send` → `SendLabOrderCommand` → Integrations
- POST `/lab-orders/{orderId}:cancel` → `CancelLabOrderCommand` → Lab
- POST `/lab-orders/{orderId}:mark-collected` → `MarkSpecimenCollectedCommand` → Lab
- POST `/lab-orders/{orderId}:mark-received` → `MarkSpecimenReceivedCommand` → Lab
- POST `/lab-orders/{orderId}:mark-complete` → `MarkLabOrderCompleteCommand` → Lab
- GET `/lab-orders/{orderId}/results` → `ListLabResultsQuery` → Lab
- GET `/lab-orders/{orderId}/results/{resultId}` → `GetLabResultQuery` → Lab

Lab reference data

- GET `/lab-tests` → `ListLabTestsQuery` → Lab
- POST `/lab-tests` → `CreateLabTestCommand` → Lab
- PATCH `/lab-tests/{testId}` → `UpdateLabTestCommand` → Lab
- DELETE `/lab-tests/{testId}` → `DeleteLabTestCommand` → Lab

Webhooks

- POST `/webhooks/lab/{provider}` → `ReceiveLabResultWebhookCommand` → Integrations
- POST `/webhooks/lab/{provider}:verify` → `VerifyLabWebhookCommand` → Integrations

Ops

- GET `/lab-orders/export` → `ExportLabOrdersQuery` → Lab
- POST `/lab-orders/bulk` → `BulkUpdateLabOrdersCommand` → Lab
- POST `/lab-orders:reconcile` → `ReconcileLabOrdersCommand` → Lab

A.10 Prescriptions & Medications (22)

- GET `/prescriptions` → `ListPrescriptionsQuery` → Pharmacy
- POST `/prescriptions` → `CreatePrescriptionCommand` → Pharmacy
- GET `/prescriptions/{rxId}` → `GetPrescriptionQuery` → Pharmacy
- PATCH `/prescriptions/{rxId}` → `UpdatePrescriptionCommand` → Pharmacy
- DELETE `/prescriptions/{rxId}` → `DeletePrescriptionCommand` → Pharmacy
- POST `/prescriptions/{rxId}:issue` → `IssuePrescriptionCommand` → Pharmacy
- POST `/prescriptions/{rxId}:cancel` → `CancelPrescriptionCommand` → Pharmacy
- POST `/prescriptions/{rxId}:dispense` → `DispensePrescriptionCommand` → Pharmacy
- GET `/prescriptions/search` → `SearchPrescriptionsQuery` → Pharmacy
- GET `/prescriptions/export` → `ExportPrescriptionsQuery` → Pharmacy

Medication catalog

- GET `/medications` → `ListMedicationsQuery` → Pharmacy
- POST `/medications` → `CreateMedicationCommand` → Pharmacy
- GET `/medications/{medId}` → `GetMedicationQuery` → Pharmacy
- PATCH `/medications/{medId}` → `UpdateMedicationCommand` → Pharmacy
- DELETE `/medications/{medId}` → `DeleteMedicationCommand` → Pharmacy
- GET `/medications/search` → `SearchMedicationsQuery` → Pharmacy

Allergies

- GET `/patients/{patientId}/allergies` → `ListAllergiesQuery` → Pharmacy
- POST `/patients/{patientId}/allergies` → `AddAllergyCommand` → Pharmacy

- DELETE `/patients/{patientId}/allergies/{allergyId}` → `RemoveAllergyCommand` → Pharmacy
- GET `/patients/{patientId}/medications` → `ListPatientMedicationsQuery` → Pharmacy

A.11 Billing: Invoices, Payments, Pricing (40)

Pricing & catalog

- GET `/services` → `ListBillableServicesQuery` → Billing
- POST `/services` → `CreateBillableServiceCommand` → Billing
- PATCH `/services/{serviceId}` → `UpdateBillableServiceCommand` → Billing
- DELETE `/services/{serviceId}` → `DeleteBillableServiceCommand` → Billing
- GET `/price-lists` → `ListPriceListsQuery` → Billing
- POST `/price-lists` → `CreatePriceListCommand` → Billing
- GET `/price-lists/{priceListId}` → `GetPriceListQuery` → Billing
- PATCH `/price-lists/{priceListId}` → `UpdatePriceListCommand` → Billing
- DELETE `/price-lists/{priceListId}` → `DeletePriceListCommand` → Billing
- PUT `/price-lists/{priceListId}/items` → `SetPriceListItemsCommand` → Billing

Invoices

- GET `/invoices` → `ListInvoicesQuery` → Billing
- POST `/invoices` → `CreateInvoiceCommand` → Billing
- GET `/invoices/{invoiceId}` → `GetInvoiceQuery` → Billing
- PATCH `/invoices/{invoiceId}` → `UpdateInvoiceCommand` → Billing
- DELETE `/invoices/{invoiceId}` → `DeleteInvoiceCommand` → Billing
- POST `/invoices/{invoiceId}:issue` → `IssueInvoiceCommand` → Billing
- POST `/invoices/{invoiceId}:void` → `VoidInvoiceCommand` → Billing
- POST `/invoices/{invoiceId}:finalize` → `FinalizeInvoiceCommand` → Billing
- GET `/invoices/{invoiceId}/items` → `ListInvoiceItemsQuery` → Billing
- POST `/invoices/{invoiceId}/items` → `AddInvoiceItemCommand` → Billing
- PATCH `/invoices/{invoiceId}/items/{itemId}` → `UpdateInvoiceItemCommand` → Billing
- DELETE `/invoices/{invoiceId}/items/{itemId}` → `RemoveInvoiceItemCommand` → Billing
- GET `/invoices/search` → `SearchInvoicesQuery` → Billing
- GET `/invoices/export` → `ExportInvoicesQuery` → Billing

Payments

- GET `/payments` → `ListPaymentsQuery` → Billing
- POST `/payments:initiate` → `InitiatePaymentCommand` → Billing
- GET `/payments/{paymentId}` → `GetPaymentQuery` → Billing
- GET `/payments/{paymentId}/status` → `GetPaymentStatusQuery` → Billing
- POST `/payments/{paymentId}:cancel` → `CancelPaymentCommand` → Billing

- POST `/payments/{paymentId}:refund` → `RefundPaymentCommand` → Billing
- POST `/payments/{paymentId}:capture` → `CapturePaymentCommand` → Billing

Reconciliation

- POST `/payments:reconcile` → `ReconcilePaymentsCommand` → Billing
- GET `/payments/reconciliation-runs` → `ListReconciliationRunsQuery` → Billing
- GET `/payments/reconciliation-runs/{runId}` → `GetReconciliationRunQuery` → Billing

Webhooks (Uzbek payments)

- POST `/webhooks/payme` → `HandlePaymeWebhookCommand` → Integrations
- POST `/webhooks/click` → `HandleClickWebhookCommand` → Integrations
- POST `/webhooks/uzum` → `HandleUzumWebhookCommand` → Integrations
- POST `/webhooks/payme:verify` → `VerifyPaymeWebhookCommand` → Integrations
- POST `/webhooks/click:verify` → `VerifyClickWebhookCommand` → Integrations
- POST `/webhooks/uzum:verify` → `VerifyUzumWebhookCommand` → Integrations

A.12 Insurance Claims (State Machine) (28)

- GET `/insurance/payers` → `ListPayersQuery` → Insurance
- POST `/insurance/payers` → `CreatePayerCommand` → Insurance
- PATCH `/insurance/payers/{payerId}` → `UpdatePayerCommand` → Insurance
- DELETE `/insurance/payers/{payerId}` → `DeletePayerCommand` → Insurance
- GET `/claims` → `ListClaimsQuery` → Insurance
- POST `/claims` → `CreateClaimCommand` → Insurance
- GET `/claims/{claimId}` → `GetClaimQuery` → Insurance
- PATCH `/claims/{claimId}` → `UpdateClaimCommand` → Insurance
- DELETE `/claims/{claimId}` → `DeleteClaimCommand` → Insurance
- GET `/claims/search` → `SearchClaimsQuery` → Insurance
- GET `/claims/export` → `ExportClaimsQuery` → Insurance

Claim state actions

- POST `/claims/{claimId}:submit` → `SubmitClaimCommand` → Insurance
- POST `/claims/{claimId}:start-review` → `StartClaimReviewCommand` → Insurance
- POST `/claims/{claimId}:approve` → `ApproveClaimCommand` → Insurance
- POST `/claims/{claimId}:deny` → `DenyClaimCommand` → Insurance
- POST `/claims/{claimId}:mark-paid` → `MarkClaimPaidCommand` → Insurance
- POST `/claims/{claimId}:reopen` → `ReopenClaimCommand` → Insurance

Attachments

- GET `/claims/{claimId}/attachments` → `ListClaimAttachmentsQuery` → Insurance
- POST `/claims/{claimId}/attachments` → `UploadClaimAttachmentCommand` → Insurance
- DELETE `/claims/{claimId}/attachments/{attachmentId}` → `DeleteClaimAttachmentCommand` → Insurance

Rules

- GET `/insurance/rules` → `ListInsuranceRulesQuery` → Insurance
- POST `/insurance/rules` → `CreateInsuranceRuleCommand` → Insurance
- PATCH `/insurance/rules/{ruleId}` → `UpdateInsuranceRuleCommand` → Insurance
- DELETE `/insurance/rules/{ruleId}` → `DeleteInsuranceRuleCommand` → Insurance

A.13 Notifications: SMS/Email/Telegram (34)

Templates

- GET `/templates` → `ListTemplatesQuery` → Notifications
- POST `/templates` → `CreateTemplateCommand` → Notifications
- GET `/templates/{templateId}` → `GetTemplateQuery` → Notifications
- PATCH `/templates/{templateId}` → `UpdateTemplateCommand` → Notifications
- DELETE `/templates/{templateId}` → `DeleteTemplateCommand` → Notifications
- POST `/templates/{templateId}:test-render` → `TestRenderTemplateCommand` → Notifications

Channels

- POST `/notifications:test/sms` → `SendTestSmsCommand` → Notifications
- POST `/notifications:test/email` → `SendTestEmailCommand` → Notifications
- POST `/notifications:test/telegram` → `SendTestTelegramCommand` → Notifications

Dispatch

- POST `/notifications` → `SendNotificationCommand` → Notifications
- GET `/notifications` → `ListNotificationsQuery` → Notifications
- GET `/notifications/{notificationId}` → `GetNotificationQuery` → Notifications
- POST `/notifications/{notificationId}:retry` → `RetryNotificationCommand` → Notifications
- POST `/notifications/{notificationId}:cancel` → `CancelNotificationCommand` → Notifications

Provider configs

- GET `/notification-providers/sms` → `ListSmsProvidersQuery` → Notifications

- PUT `/notification-providers/sms` → `SetSmsProvidersPriorityCommand` → Notifications
- GET `/notification-providers/email` → `GetEmailProviderQuery` → Notifications
- PUT `/notification-providers/email` → `SetEmailProviderCommand` → Notifications
- GET `/notification-providers/telegram` → `GetTelegramProviderQuery` → Notifications
- PUT `/notification-providers/telegram` → `SetTelegramProviderCommand` → Notifications

SMS providers (Uz)

- POST `/integrations/eskiz:send` → `SendEskizSmsCommand` → Integrations
- POST `/integrations/playmobile:send` → `SendPlayMobileSmsCommand` → Integrations
- POST `/integrations/textup:send` → `SendTextUpSmsCommand` → Integrations

Telegram

- POST `/webhooks/telegram` → `HandleTelegramWebhookCommand` → Integrations
- POST `/telegram/bot:broadcast` → `BroadcastTelegramCommand` → Notifications
- POST `/telegram/bot:sync` → `SyncTelegramBotCommand` → Integrations

Email

- POST `/email:send` → `SendEmailCommand` → Notifications
- GET `/email/events` → `ListEmailEventsQuery` → Notifications

A. 14 Integrations Hub (Configs, Tokens, Webhooks) (36)

Integration registry

- GET `/integrations` → `ListIntegrationsQuery` → Integrations
- GET `/integrations/{integrationKey}` → `GetIntegrationQuery` → Integrations
- POST `/integrations/{integrationKey}:enable` → `EnableIntegrationCommand` → Integrations
- POST `/integrations/{integrationKey}:disable` → `DisableIntegrationCommand` → Integrations

Credentials per tenant

- GET `/integrations/{integrationKey}/credentials` → `GetIntegrationCredentialsQuery` → Integrations
- PUT `/integrations/{integrationKey}/credentials` → `UpsertIntegrationCredentialsCommand` → Integrations
- DELETE `/integrations/{integrationKey}/credentials` → `DeleteIntegrationCredentialsCommand` → Integrations

Health & diagnostics

- GET `/integrations/{integrationKey}/health` → `IntegrationHealthQuery` → Integrations
- POST `/integrations/{integrationKey}:test-connection` → `TestIntegrationConnectionCommand` → Integrations
- GET `/integrations/{integrationKey}/logs` → `ListIntegrationLogsQuery` → Integrations

Webhook management

- GET `/integrations/{integrationKey}/webhooks` → `ListIntegrationWebhooksQuery` → Integrations
- POST `/integrations/{integrationKey}/webhooks` → `CreateIntegrationWebhookCommand` → Integrations
- DELETE `/integrations/{integrationKey}/webhooks/{webhookId}` → `DeleteIntegrationWebhookCommand` → Integrations
- POST `/integrations/{integrationKey}/webhooks/{webhookId}:rotate-secret` → `RotateWebhookSecretCommand` → Integrations

OAuth token stores

- GET `/integrations/{integrationKey}/tokens` → `ListIntegrationTokensQuery` → Integrations
- POST `/integrations/{integrationKey}/tokens:refresh` → `RefreshIntegrationTokensCommand` → Integrations
- DELETE `/integrations/{integrationKey}/tokens/{tokenId}` → `RevokeIntegrationTokenCommand` → Integrations

Uzbekistan add-ons (optional)

- POST `/integrations/myid:verify` → `VerifyMyIdCommand` → Integrations
- POST `/webhooks/myid` → `HandleMyIdWebhookCommand` → Integrations
- POST `/integrations/eimzo:sign` → `CreateEImzoSignRequestCommand` → Integrations
- POST `/webhooks/eimzo` → `HandleEImzoWebhookCommand` → Integrations

A.15 Audit & Compliance (18)

- GET `/audit/events` → `ListAuditEventsQuery` → Audit
- GET `/audit/events/{eventId}` → `GetAuditEventQuery` → Audit
- GET `/audit/export` → `ExportAuditEventsQuery` → Audit
- GET `/audit/retention` → `GetAuditRetentionQuery` → Audit
- PUT `/audit/retention` → `UpdateAuditRetentionCommand` → Audit
- GET `/audit/object/{objectType}/{objectId}` → `GetObjectAuditQuery` → Audit
- GET `/compliance/pii-fields` → `ListPiiFieldsQuery` → Compliance

- PUT `/compliance/pii-fields` → `SetPiiFieldsCommand` → Compliance
- POST `/compliance/pii:rotate-keys` → `RotatePiiKeysCommand` → Compliance
- POST `/compliance/pii:re-encrypt` → `ReEncryptPiiCommand` → Compliance
- GET `/consents` → `ListConsentsQuery` → Compliance
- GET `/consents/{consentId}` → `GetConsentQuery` → Compliance
- GET `/data-access-requests` → `ListDataAccessRequestsQuery` → Compliance
- POST `/data-access-requests` → `CreateDataAccessRequestCommand` → Compliance
- POST `/data-access-requests/{requestId}:approve` → `ApproveDataAccessRequestCommand` → Compliance
- POST `/data-access-requests/{requestId}:deny` → `DenyDataAccessRequestCommand` → Compliance
- GET `/data-access-requests/{requestId}` → `GetDataAccessRequestQuery` → Compliance
- GET `/compliance/reports` → `ListComplianceReportsQuery` → Compliance

A.16 Observability, Health, Admin Ops (22)

- GET `/health` → `HealthQuery` → Ops
- GET `/ready` → `ReadinessQuery` → Ops
- GET `/live` → `LivenessQuery` → Ops
- GET `/metrics` → `MetricsQuery` → Ops
- GET `/version` → `VersionQuery` → Ops

Admin ops

- POST `/admin/cache:flush` → `FlushCacheCommand` → Ops
- POST `/admin/cache:rebuild` → `RebuildCachesCommand` → Ops
- GET `/admin/jobs` → `ListJobsQuery` → Ops
- POST `/admin/jobs/{jobId}:retry` → `RetryJobCommand` → Ops
- GET `/admin/kafka/lag` → `GetKafkaLagQuery` → Ops
- POST `/admin/kafka:replay` → `ReplayKafkaEventsCommand` → Ops
- GET `/admin/outbox` → `ListOutboxQuery` → Ops
- POST `/admin/outbox:drain` → `DrainOutboxCommand` → Ops
- POST `/admin/outbox/{outboxId}:retry` → `RetryOutboxItemCommand` → Ops
- GET `/admin/logging/pipelines` → `ListLoggingPipelinesQuery` → Ops
- POST `/admin/logging:pipeline-reload` → `ReloadLoggingPipelinesCommand` → Ops
- GET `/admin/feature-flags` → `ListFeatureFlagsQuery` → Ops
- PUT `/admin/feature-flags` → `SetFeatureFlagsCommand` → Ops
- GET `/admin/rate-limits` → `GetRateLimitsQuery` → Ops
- PUT `/admin/rate-limits` → `UpdateRateLimitsCommand` → Ops
- GET `/admin/config` → `GetRuntimeConfigQuery` → Ops
- POST `/admin/config:reload` → `ReloadRuntimeConfigCommand` → Ops

A.17 Reference Data & Search (18)

- GET `/reference/currencies` → `ListCurrenciesQuery` → Shared
- GET `/reference/countries` → `ListCountriesQuery` → Shared
- GET `/reference/languages` → `ListLanguagesQuery` → Shared
- GET `/reference/diagnosis-codes` → `ListDiagnosisCodesQuery` → Shared
- GET `/reference/procedure-codes` → `ListProcedureCodesQuery` → Shared
- GET `/reference/insurance-codes` → `ListInsuranceCodesQuery` → Shared
- GET `/search/global` → `GlobalSearchQuery` → Shared
- GET `/search/patients` → `SearchPatientsQuery` → Patient
- GET `/search/providers` → `SearchProvidersQuery` → Provider
- GET `/search/appointments` → `SearchAppointmentsQuery` → Scheduling
- GET `/search/invoices` → `SearchInvoicesQuery` → Billing
- GET `/search/claims` → `SearchClaimsQuery` → Insurance
- GET `/reports` → `ListReportsQuery` → Reporting
- POST `/reports` → `CreateReportCommand` → Reporting
- GET `/reports/{reportId}` → `GetReportQuery` → Reporting
- POST `/reports/{reportId}:run` → `RunReportCommand` → Reporting
- GET `/reports/{reportId}/download` → `DownloadReportQuery` → Reporting
- DELETE `/reports/{reportId}` → `DeleteReportCommand` → Reporting

A.18 Totals

Approximate count by section: - Auth & Identity: 16 - Tenants: 12 - Clinics & Locations: - Users/Roles/Permissions: 38 - Patients: 30 - Providers & Availability: 34 - Scheduling/Appointments: 44 - Treatment & Encounters: 32 - Labs: 28 - Prescriptions: 22 - Billing & Payments: 40 - Insurance Claims: 28 - Notifications: 34 - Integrations Hub: 36 - Audit & Compliance: 18 - Observability/Admin Ops: 22 - Reference Data & Search: 18

Total inventory: ~ 478 lines listed here, with ~ 280 – 320 **distinct endpoints** depending on whether optional endpoints (MyID/E-IMZO, reporting) are enabled.

Appendix B — ADR Template

`/docs/adr/000-template.md` - Context - Decision - Alternatives - Consequences - Migration Plan

---

Appendix C — Done Definition

A feature is “done” when: - OpenAPI updated - Tests added - Observability updated - ADR if violations